

Assignment 4 — Chapters 4 & 5

Web Services, REST APIs & Web Security

Part A — Short Answer Questions

Q1. Define Web Service.

A Web Service is a standardized way of enabling communication between two electronic devices or software applications over a network (typically the internet). It allows different applications, often built on different platforms and languages, to exchange data using open protocols such as HTTP, XML, SOAP, or JSON. Web services expose their functionality through well-defined interfaces (e.g., APIs) so that other programs can discover and use them.

Q2. What is the difference between SOAP and REST?

SOAP (Simple Object Access Protocol) is a protocol with strict standards; it uses XML for message format and typically relies on WSDL for service description. It supports WS-Security and built-in error handling.

REST (Representational State Transfer) is an architectural style, not a protocol. It uses standard HTTP methods, supports multiple data formats (JSON, XML, etc.), is stateless, lightweight, and generally faster and easier to implement than SOAP. REST is the dominant choice for modern web and mobile APIs.

Q3. Write any two examples of HTTP methods.

GET — Retrieves/reads data from a server (e.g., fetching a list of students).

POST — Sends new data to the server to create a resource (e.g., adding a new student record).

Q4. What is an API endpoint?

An API endpoint is a specific URL (Uniform Resource Locator) at which an API can be accessed by a client application. Each endpoint corresponds to a particular resource or function. For example, `https://api.example.com/students` could be an endpoint that returns student data.

Q5. Define WSDL.

WSDL (Web Services Description Language) is an XML-based language used to describe the functionality offered by a web service. It specifies the service location, available operations, the message formats (input/output), and the communication protocols used. WSDL is primarily associated with SOAP-based web services.

Q6. What is meant by stateless communication?

Stateless communication means that each request from a client to a server must contain all the information the server needs to understand and process that request. The server does not store any session information about the client between requests. Every request is treated independently.

REST APIs follow this principle.

Q7. Define Authentication.

Authentication is the process of verifying the identity of a user, device, or system. It answers the question "*Who are you?*". Common methods include username/password credentials, biometrics, multi-factor authentication (MFA), and token-based systems like JWT.

Q8. Define Authorization.

Authorization is the process of determining what an authenticated user is allowed to do. It answers the question "*What are you permitted to access?*". For example, a regular user may only view data, while an admin can also create, update, or delete it. Authorization always comes after authentication.

Q9. What is HTTPS?

HTTPS (HyperText Transfer Protocol Secure) is the secure version of HTTP. It uses TLS/SSL encryption to protect data transmitted between the client (browser) and the server, ensuring confidentiality, integrity, and authenticity. HTTPS prevents eavesdropping and man-in-the-middle attacks and is indicated by a padlock icon in the browser address bar.

Q10. What is XSS attack?

XSS (Cross-Site Scripting) is a web security vulnerability in which an attacker injects malicious client-side scripts (usually JavaScript) into web pages viewed by other users. When the victim's browser executes the injected script, it can steal cookies, session tokens, or personal data, and can even redirect users to malicious sites.

Part B — Long Answer Questions

Q1. Explain the working principle of a RESTful Web Service.

REST (Representational State Transfer) is an architectural style for designing networked applications. A RESTful web service works on the following principles:

- **Client-Server Architecture:** The client and server are separate entities. The client sends requests and the server processes them and returns responses.
- **Statelessness:** Each request from the client contains all necessary information. The server does not maintain client state between requests.
- **Uniform Interface:** Resources are identified by URIs. Standard HTTP methods (GET, POST, PUT, DELETE) are used to perform operations on these resources.
- **Resource-Based:** Everything is treated as a resource (e.g., a student, a course). Each resource has a unique URI.
- **Representation:** Resources are represented in formats like JSON or XML. The client and server exchange these representations.
- **Cacheability:** Responses can be marked as cacheable or non-cacheable, improving performance.

Typical flow: The client sends an HTTP request (e.g., GET /api/students) → the server processes it → the server returns an HTTP response with a status code (e.g., 200 OK) and data (e.g., JSON array of students).

Q2. Explain the role of JSON in REST APIs.

JSON (JavaScript Object Notation) is the most commonly used data interchange format in REST APIs. Its role includes:

- **Data Exchange Format:** JSON serves as the standard format for sending data in request bodies and receiving data in responses.
- **Lightweight & Readable:** JSON is text-based, human-readable, and much lighter than XML, making it efficient for network transmission.
- **Language Independent:** Although derived from JavaScript, JSON can be parsed by virtually every programming language (Python, Java, C#, etc.).
- **Structured Data:** JSON supports objects (key-value pairs) and arrays, allowing representation of complex nested data structures.

Example: `{ "id": 1, "name": "Alice", "grade": "A" }`

Q3. What is API documentation? Why is it important?

API documentation is a technical reference that describes how to use and integrate with an API. It typically includes available endpoints, HTTP methods, request/response formats, authentication requirements, error codes, and usage examples.

Importance:

- Helps developers understand how to correctly consume the API without reading source code.
- Reduces integration time and errors.
- Serves as a contract between API provider and consumer.
- Facilitates testing and debugging.

- Tools like Swagger/OpenAPI and Postman can auto-generate interactive documentation.

Q4. Explain the difference between session-based authentication and token-based authentication.

Session-Based Authentication:

- After login, the server creates a session and stores session data on the server side.
- A session ID is sent to the client (usually via a cookie).
- On each subsequent request, the client sends the session ID; the server looks it up to verify the user.
- The server is stateful — it must maintain session storage.

Token-Based Authentication (e.g., JWT):

- After login, the server generates a signed token (e.g., JWT) and sends it to the client.
- The client stores the token (in memory, cookie, etc.) and sends it in the Authorization header with each request.
- The server verifies the token's signature without needing stored session data.
- The server is stateless — no session storage is needed, making it highly scalable.

Key difference: Session-based is server-stateful; token-based is server-stateless and better suited for distributed and mobile applications.

Q5. What is SQL Injection? Explain with a simple example.

SQL Injection is a code-injection attack in which an attacker inserts malicious SQL statements into input fields that are directly included in database queries. If the application does not properly sanitize user input, the attacker can manipulate the query to access, modify, or delete data.

Example:

A login form constructs a query like:

```
SELECT * FROM users WHERE username = '[input]' AND password = '[input]';
```

If the attacker enters the username as: ' OR 1=1 --

The query becomes:

```
SELECT * FROM users WHERE username = '' OR 1=1 --' AND password = '';
```

Since 1=1 is always true and -- comments out the rest, the query returns all users, bypassing authentication.

Prevention: Use parameterized queries (prepared statements), input validation, and ORM frameworks.

Q6. Explain CSRF attack in short.

CSRF (Cross-Site Request Forgery) is an attack that tricks an authenticated user into unknowingly submitting a malicious request to a web application where they are already logged in.

How it works: The victim is logged into a legitimate site (e.g., a banking site). The attacker crafts a malicious link or hidden form on a different website. When the victim visits the attacker's page, the browser automatically sends the victim's session cookie along with the forged request to the banking site, which processes it as a legitimate action (e.g., transferring funds).

Prevention:

- Use anti-CSRF tokens (unique, unpredictable tokens included in forms).
- Use the SameSite cookie attribute.
- Verify the Origin/Referer headers on the server.

Q7. What are secure cookies? Mention their types/flags.

Secure cookies are HTTP cookies configured with special flags/attributes that enhance their security and limit exposure to attacks.

Important cookie flags:

- **Secure:** The cookie is only sent over HTTPS connections, preventing interception over unencrypted HTTP.
- **HttpOnly:** The cookie cannot be accessed by client-side JavaScript (document.cookie), mitigating XSS-based cookie theft.
- **SameSite:** Controls whether the cookie is sent with cross-site requests. Values: *Strict* (never sent cross-site), *Lax* (sent with top-level navigations), *None* (always sent, requires Secure flag).
- **Domain:** Restricts which domains can receive the cookie.
- **Path:** Restricts the cookie to a specific URL path.
- **Expires / Max-Age:** Sets the cookie's lifetime. Without these, the cookie is a session cookie deleted when the browser closes.

Q8. Explain why input validation is important in web security.

Input validation is the process of checking and sanitizing all data received from users before processing it. It is a critical first line of defense in web security.

Why it matters:

- **Prevents Injection Attacks:** Without validation, attackers can inject SQL, JavaScript (XSS), or OS commands into input fields.
- **Prevents Data Corruption:** Ensures that only correctly formatted data enters the database (e.g., email fields contain valid emails).
- **Reduces Attack Surface:** Rejecting unexpected input early stops many attack vectors before they reach deeper application logic.
- **Protects Business Logic:** Malformed inputs can cause unexpected application behavior or crashes.

Best practices: Validate on both client side and server side; use whitelisting (allow known-good patterns) over blacklisting; sanitize and escape output; use framework-provided validation libraries.

Q9. Explain the working of JWT authentication with flow diagram.

JWT (JSON Web Token) is a compact, URL-safe token used for securely transmitting information between parties as a JSON object. It is widely used for stateless authentication in REST APIs.

Structure of a JWT: A JWT consists of three parts separated by dots: Header.Payload.Signature

- **Header:** Contains the token type (JWT) and signing algorithm (e.g., HS256).
- **Payload:** Contains claims — user data such as user ID, role, and expiration time.
- **Signature:** Created by signing the encoded header and payload with a secret key. Used to verify the token's integrity.

Authentication Flow:

1. The client sends login credentials (username & password) to the server.
2. The server verifies the credentials against the database.
3. If valid, the server generates a JWT signed with a secret key and returns it to the client.
4. The client stores the JWT (e.g., in memory or an HttpOnly cookie).
5. For each subsequent request, the client sends the JWT in the Authorization header:
Authorization: Bearer <token>.
6. The server verifies the JWT signature. If valid, it processes the request; otherwise, it returns 401 Unauthorized.

Step	Actor	Action
1	Client	Sends username & password to /login
2	Server	Validates credentials against DB
3	Server	Generates JWT (Header.Payload.Signature)
4	Server → Client	Returns JWT in response
5	Client	Stores JWT; sends it in Authorization header
6	Server	Verifies JWT signature on each request
7	Server	Grants or denies access based on token validity

Q10. Design 5 REST endpoints for a Student Management API.

#	Method	Endpoint	Description
1	GET	/api/students	Retrieve all students
2	GET	/api/students/{id}	Retrieve a single student by ID
3	POST	/api/students	Create a new student
4	PUT	/api/students/{id}	Update an existing student
5	DELETE	/api/students/{id}	Delete a student by ID

Sample Response Formats:

GET /api/students → 200 OK

```
[ { "id": 1, "name": "Alice", "age": 20, "grade": "A" },  
  { "id": 2, "name": "Bob", "age": 22, "grade": "B" } ]
```

GET /api/students/1 → 200 OK

```
{ "id": 1, "name": "Alice", "age": 20, "grade": "A" }
```

POST /api/students (Request Body: { "name": "Charlie", "age": 21, "grade": "B+" }) → 201 Created

```
{ "id": 3, "name": "Charlie", "age": 21, "grade": "B+" }
```

PUT /api/students/1 (Request Body: { "name": "Alice", "age": 21, "grade": "A+" }) → 200 OK

```
{ "id": 1, "name": "Alice", "age": 21, "grade": "A+" }
```

DELETE /api/students/1 → 200 OK

```
{ "message": "Student deleted successfully" }
```

Q11. Why should sensitive data never be stored in localStorage?

localStorage is a browser storage mechanism that persists data as plain text with no expiration. Storing sensitive data (tokens, passwords, personal information) in localStorage is dangerous for several reasons:

- **Vulnerable to XSS:** Any JavaScript running on the page can access localStorage via `localStorage.getItem()`. If the site has an XSS vulnerability, an attacker's script can steal all stored data.
- **No Encryption:** Data is stored as plain text, readable by anyone with access to the browser's developer tools.
- **No Expiry:** Unlike cookies, localStorage has no built-in expiry mechanism. Data persists indefinitely unless explicitly removed.
- **Shared Across Tabs:** All tabs/windows of the same origin share the same localStorage, increasing exposure.
- **No HttpOnly Equivalent:** Unlike cookies, there is no flag to prevent JavaScript access.

Better alternatives: Use HttpOnly + Secure cookies for tokens, or store sensitive data only in server-side sessions.

Q12. Explain the difference between encryption and hashing with real examples.

Encryption:

Encryption is a reversible process that converts plaintext into ciphertext using an algorithm and a key. The original data can be recovered using the correct decryption key. It is used when data needs to be read again later (e.g., sending a secure email, HTTPS communication, encrypting files).

Example: AES encryption of the message "Hello" with a secret key produces ciphertext like `U2FsSGVkbX1...`. Using the same key, the ciphertext can be decrypted back to "Hello".

Hashing:

Hashing is a one-way process that converts input into a fixed-length string (hash/digest). It cannot be reversed — you cannot recover the original data from the hash. The same input always produces the same hash. It is used for data integrity verification and password storage.

Example: SHA-256 hash of "password123" → `ef92b778...`. There is no key to reverse this.

Feature	Encryption	Hashing
Reversible?	Yes (with key)	No (one-way)
Key required?	Yes	No
Output length	Varies	Fixed length
Use case	Secure data transmission	Password storage, integrity checks
Example algorithms	AES, RSA	SHA-256, bcrypt